

Bracing the SWAP hydrological model for the future with the pySWAP Python package

Mateusz Zawadzki¹[¶], Mark van de Brink², and Marijke Huysmans¹

¹ Vrije Universiteit Brussel, Belgium ² Wageningen University and Research, the Netherlands [¶] Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 13 April 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))[¶].

Summary

pySWAP is an open-source Python package for creating, running, analyzing, and sharing Soil-Water-Plant-Atmosphere (SWAP) hydrological models. SWAP calculates crop water requirements and simulates interactions between water balance components such as evaporation and groundwater storage (Kroes et al., 2017). The primary purpose of pySWAP is to eliminate the need for plain-text input files that SWAP requires, enabling smoother automation and seamless integration of modeling workflows with the variety of analytical tools the Python ecosystem offers. pySWAP was first presented as a concept at the EGU General Assembly in 2024 (Zawadzki, 2024) and has since matured into a production-ready package with documentation, a test suite, and active research use.

Statement of Need

SWAP has played an important role in the advancement of agrohydrological research over the last 50 years and has been widely applied in studies focusing on agricultural water management and irrigation optimization (Heinen et al., 2024; Kroes et al., 2017). The model is particularly valuable in agricultural applications due to its integration with the dynamic crop growth model WOFOST (Boogaard et al., 2014), which enables the simulation of crop responses to varying water and nutrient conditions. SWAP is part of tools used for policy making in the European Union (e.g. FOCUSPEARL (Van Den Berg et al., 2019)) and the Netherlands (e.g. WaterVision Agriculture (Hack-ten Broeke et al., 2019)). SWAP code and compiler settings are open-source under the GNU General Public License. Version 4.2.0 is distributed with supplementary materials and documentation via swap.wur.nl.

Like many Fortran programs, SWAP requires a set of plain-text ASCII files with custom extensions. At execution, the TTUTIL library (Kraalingen & Rappoldt, 2000) parses these files and loads variables into memory. After completion, the program writes output files to disk. Although this workflow is sufficient for small projects, it becomes cumbersome when managing multiple models — particularly for sensitivity analysis, scenario analysis, or auto-calibration procedures across multiple field sites on high-performance computing systems. As the community grows and research questions become more complex, each modeller inevitably develops their own ad-hoc wrapper, creating inconsistency and poor sharability of workflows.

State of the Field

The only published tools for SWAP input/output handling are currently written in R (e.g., rSWAP (Shore, 2023) and SWAPTools, distributed with SWAP 4.2.0). Prior to pySWAP, no structured, tested, and peer-reviewed Python library for SWAP model interaction existed. Given that Python has become the dominant language for scientific data analysis and hydrological

40 modeling workflows, this gap limits the accessibility of SWAP and therefore the potential for
41 community-driven model development.

42 The precedent for wrapping Fortran-based hydrological models in Python is well established —
43 most notably by flopy (Bakker et al., 2025; Hughes et al., 2024), which demonstrated that an
44 object-oriented Python interface to MODFLOW significantly lowers barriers to automation,
45 reproducibility, and community adoption. Beyond file handling, a native Python interface
46 unlocks the full scientific ecosystem — from statistical analysis and machine learning to
47 visualization and model coupling — enabling modelers to integrate SWAP directly into their
48 broader research workflows without manual data exchange (Hughes et al., 2022). The pySWAP
49 architecture is also designed to anticipate the evolution of SWAP itself: as the SWAP fortran
50 codebase is currently maturing toward a callable library interface, tools such as f2py will allow
51 direct in-memory coupling, eliminating file-based execution entirely.

52 Software design

53 pySWAP's object-oriented design is inspired by flopy (Bakker et al., 2025; Hughes et al.,
54 2024), with functionality split into modules for intuitive access. The design is guided by three
55 principles: (1) map SWAP's conceptual structure directly onto Python objects so the model is
56 readable and auditable without consulting raw input files, (2) leverage the Python scientific
57 ecosystem (numpy, pandas, pydantic) rather than reimplementing standard functionality, and
58 (3) maintain separation between model definition, execution, and result analysis for easier
59 extensibility.

60 Users primarily interact with the components and model modules. Each component (e.g.,
61 meteorology, crop) groups co-dependent attributes corresponding to sections of the original
62 SWAP input files, with input validation handled via pydantic to catch incorrect model setups
63 before any files are written or the model is executed. Once assembled into a Model object,
64 these components build input files directory, call the SWAP executable, and return a structured
65 Result object used in further analysis.

66 Modelling workflow

67 We recommend using the pyswap CLI to start new projects. Running pyswap init prompts
68 users for project metadata and generates a structured directory with a default Jupyter notebook
69 and pixi.toml file for dependency management. To get started with pyswap, users can consult
70 the documentation's quick start section or preview the interactive Binder environment with
71 documentation notebooks. The following example illustrates a minimal model setup:

```
import pyswap as ps
from pyswap import testcase

# Reading CSV data:
# metfile_from_csv will return a MetFile object, a file class
meteo_data = ps.components.meteorology.metfile_from_csv(
    metfil="283.met", csv_path=testcase.get_path("hupselbrook", "met")
)

# Defining components:
# Meteorology object is an example of a section class
meteo = ps.components.meteorology.Meteorology(
    lat=52.0,
    alt=21.0,
    swetr=0,
    metfile=meteo_data,
    swdivide=1,
```

```
swmetdetail=0,  
altw=10.0,  
angstroma=0.25,  
angstromb=0.5,  
)  
  
# Composing Model class:  
  
ml = ps.Model(  
    meteorology=meteo,  
    # Other elements...  
)  
  
result = ml.run()
```

Integrations

WOFOST Crop Parameters Database

The package provides access to databases of calibrated crop parameters (Wit & Burggraaff, 2025). Users can inspect parameter content, modify them, and update their pyswap crop settings objects accordingly.

HDF5 database

pyswap models and results can be stored in HDF5 format, a versatile scientific computing standard. Each model is saved (optionally with results) as a pickled object with metadata.

Belgian and Dutch soil databases

Packages `dovvms` (Zawadzki, 2025a), `simplesoilprofile` (Zawadzki, 2025b), and `DutchSoils` (Brink, 2025) provide quick access to Belgian and Dutch soil databases, enabling workflow automation and reducing potential errors.

Testing

The current version has 84% code coverage using the `pytest` testing framework. Tests target specific features and validate model results against known benchmarks within acceptable error margins.

CI/CD

The package implements robust continuous integration and deployment. Package and documentation builds are tested before each release, and documentation is automatically deployed with each new software version.

Research impact statement

pyswap has been successfully used in two research projects, with publications in preparation and will be adopted in a newly started European project:

- [Project GROW](#) - modeling soil moisture conditions at an experimental field where treated wastewater is reused for subirrigation,

- [WaterScape](#) - determining the role of soil-water-plant-atmosphere interactions in shallow groundwater and rooting zones for local water availability and land-use practices, and
- [AquiCirc](#) - funded under the Water4All initiative, focused on Managed Aquifer Recharge for a Circular Water Future, where pyswap will be used to simulate vadose zone dynamics under recharge scenarios.

Since its public introduction at EGU 2024 ([Zawadzki, 2024](#)), the package has received 22 GitHub stars and grown through 21 releases. It has attracted contributions and community feedback from SWAP modellers across the Netherlands, Belgium, and beyond — via GitHub issues and direct contact. As pyswap lowers the barrier to adoption and connects SWAP to the broader Python ecosystem, it is expected to expand the active user base — accelerating community-driven development and making the model more robust through collective feedback and contribution. Active development continues, with planned extensions for spatial support, sensitivity analysis, and improved multicore processing.

AI usage disclosure Generative AI tools (GPT Codex and Claude Sonnet) were used in the development of this software for troubleshooting, targeted implementation assistance, certain routine tasks such as CI/CD pipeline development and brainstorming technical solutions; all final design decisions were made by the authors and AI-generated contributions were manually reviewed throughout. Claude Sonnet was additionally used as a writing aid during manuscript restructuring. All scientific content and conclusions remain the sole responsibility of the authors.

Acknowledgements

The authors would like to thank all those who gave valuable feedback to this work at conferences and (face-to-face) meetings. Thanks to Ali Mehmandoostkotlar, Erika Lucia Rodriguez Lache and Sarah Garré from the Flanders Research Institute for Agriculture (ILVO) who supported the initiative at its early stages. Moreover, the authors would like to thank the broad SWAP users community for their suggestions, constructive criticism and, above all, their contributions to pyswap.

Funding

This work has been funded by the Interdisciplinary Research Project funding, an internal grant awarded to interdisciplinary research teams at the Vrije Universiteit Brussel, and WaterScape, a project funded by the Dutch Research Council, which explores ways to transform the Dutch water system in response to climate change.

References

- Bakker, M., Post, V., Hughes, J. D., Langevin, C. D., White, J. T., Leaf, A. T., Paulinski, S. R., Bellino, J. C., Morway, E. D., Toews, M. W., Larsen, J. D., Fienen, M. N., Starn, J. J., Brakenhoff, D. A., & Bonelli, W. P. (2025). *Flopy v3.10.0.dev5*. U.S. Geological Survey Software Release. <https://doi.org/10.5066/F7BK19FH>
- Boogaard, H. L., Wit, A. J. W. de, Roller, J. A. te, & Diepen, C. A. van. (2014). *WOFOST Control Centre 2.1 and WOFOST 7.1.7*. Alterra, Wageningen University & Research Centre.
- Brink, M. van den. (2025). *DutchSoils: Python package to get soil data from the Dutch Soil Map, Staring series and BOFEK clustering*. <https://github.com/markvdbrink/dutchsoils>; GitHub.

- 140 Hack-ten Broeke, M. J. D., Mulder, H. M., Bartholomeus, R. P., Van Dam, J. C., Holshof, G.,
 141 Hoving, I. E., Walvoort, D. J. J., Heinen, M., Kroes, J. G., Van Bakel, P. J. T., Supit, I.,
 142 De Wit, A. J. W., & Ruijtenberg, R. (2019). Quantitative land evaluation implemented in
 143 Dutch water management. *Geoderma*, 338, 536–545. [https://doi.org/10.1016/j.geoderma.](https://doi.org/10.1016/j.geoderma.2018.11.002)
 144 [2018.11.002](https://doi.org/10.1016/j.geoderma.2018.11.002)
- 145 Heinen, M., Mulder, M., Van Dam, J., Bartholomeus, R., De Jong Van Lier, Q., De Wit, J.,
 146 De Wit, A., & Hack - Ten Broeke, M. (2024). SWAP 50 years: Advances in modelling
 147 soil-water-atmosphere-plant interactions. *Agricultural Water Management*, 298, 108883.
 148 <https://doi.org/10.1016/j.agwat.2024.108883>
- 149 Hughes, J. D., Langevin, C. D., Paulinski, S. R., Larsen, J. D., & Brakenhoff, D. (2024). FloPy
 150 workflows for creating structured and unstructured MODFLOW models. *Groundwater*,
 151 62(1), 124–139. <https://doi.org/10.1111/gwat.13327>
- 152 Hughes, J. D., Russcher, M. J., Langevin, C. D., Morway, E. D., & McDonald, R. R. (2022).
 153 The MODFLOW Application Programming Interface for simulation control and software
 154 interoperability. *Environmental Modelling & Software*, 148, 105257. [https://doi.org/10.](https://doi.org/10.1016/j.envsoft.2021.105257)
 155 [1016/j.envsoft.2021.105257](https://doi.org/10.1016/j.envsoft.2021.105257)
- 156 Kraalingen, D. W. G. van, & Rappoldt, C. (2000). *Reference manual of the FORTRAN*
 157 *utility library TTUTIL v. 4* (Report No. 5). Plant Research International. [https:](https://swap.wur.nl/TTutil/427/Ttutil4_13.pdf)
 158 [//swap.wur.nl/TTutil/427/Ttutil4_13.pdf](https://swap.wur.nl/TTutil/427/Ttutil4_13.pdf)
- 159 Kroes, J. G., Van Dam, J. C., Bartholomeus, R. P., Groenendijk, P., Heinen, M., Hendriks,
 160 R. F. A., Mulder, H. M., Supit, I., & Van Walsum, P. E. V. (2017). *SWAP version 4*.
 161 Wageningen Environmental Research. <https://doi.org/10.18174/416321>
- 162 Shore, M. (2023). *Rswap - an r interface for the SWAP model*. [https://doi.org/10.5281/](https://doi.org/10.5281/zenodo.7795152)
 163 [zenodo.7795152](https://doi.org/10.5281/zenodo.7795152)
- 164 Van Den Berg, F., Tiktak, A., Van Kraalingen, D. W. G., & Boesten, J. T. T. I. (2019).
 165 *User manual for FOCUSPEARL: Version 5.5.5* (No. 164). WOT Natuur & Milieu.
 166 <https://doi.org/10.18174/509006>
- 167 Wit, A. de, & Burggraaff, O. (2025). *WOFOST_crop_parameters: Parameter sets for the*
 168 *WOFOST crop simulation model implemented in YAML*. GitHub. [https://github.com/](https://github.com/ajwdewit/WOFOST_crop_parameters)
 169 [ajwdewit/WOFOST_crop_parameters](https://github.com/ajwdewit/WOFOST_crop_parameters)
- 170 Zawadzki, M. (2024). pySWAP: Python wrapper for SWAP hydrological model. *EGU General*
 171 *Assembly Conference Abstracts*, 19745.
- 172 Zawadzki, M. (2025a). *dovwms: Python package to retrieve data from soil texture WMS layers*
 173 *from Databank Ondergrond Vlaanderen (DOV)*. [https://github.com/zawadzki/](https://github.com/zawadzki/dovwms)
 174 [dovwms](https://github.com/zawadzki/dovwms); GitHub. <https://doi.org/10.5281/zenodo.17458122>
- 175 Zawadzki, M. (2025b). *simplesoilprofile: Python package to work with soil profiles*. [https:](https://github.com/zawadzki/simplesoilprofile)
 176 [//github.com/zawadzki/simplesoilprofile](https://github.com/zawadzki/simplesoilprofile); GitHub.